

WhatsApp AI Platform

Backend Problem-Solving & Implementation Roadmap

Multimodal Agentic WhatsApp AI Automation Platform — A phased, dependency-ordered plan for a junior developer working under senior guidance

Document Type	Engineering Task Plan & Problem-Solving Roadmap (not source code)
Audience	Junior Backend Developer, reviewed by Senior Backend/AI Engineer
Scope	12 phases (Phase 0 – Phase 11), dependency-ordered
Source Input	Backend audit of reported remaining items (treated as a starting point, not fact)
Core Instruction	Verify the repository before implementing anything in this document

Table of Contents

Table of Contents	2
How To Use This Document	4
The Development Cycle Used For Every Task	4
How Each Phase Is Structured	4
PHASE 0 — Repository and System Verification	6
Verification Checklist	6
System areas to verify:	6
Reported Items to Classify	6
Definition of Done for Phase 0	7
PHASE 1 — Infrastructure Verification	8
Exact Checks	8
Mandatory vs. Optional Services	9
Definition of Done for Phase 1	9
PHASE 2 — Conversation History Loading	10
PHASE 3 — RAG Knowledge Ingestion	14
PHASE 4 — Knowledge Source Upload API	18
PHASE 5 — WhatsApp Document Processing	21
PHASE 6 — Workspace / Admin API	25
PHASE 7 — CRM Synchronization	28
PHASE 8 — MCP Architecture and External Tool Integration	31
PHASE 9 — Ollama Provider (Optional)	35
PHASE 10 — HuggingFace Provider (Optional)	38

PHASE 11 — Testing, Security, Observability, and Production Hardening	41
Master Checklist	45
Priority & Status Table	48
START HERE	50
1. Verify the repository against this document's assumptions	50
2. Verify infrastructure is running	50
3. Confirm which mandatory external credentials are configured	50
4. Read PHASE 2 in full before writing any code	50
5. Apply the UNDERSTAND → INSPECT → DESIGN → IMPLEMENT → TEST → VERIFY → FIX → DOCUMENT cycle to the very first task	50

How To Use This Document

This document turns a backend audit into a practical, dependency-ordered implementation plan. It is written for a junior developer who will implement the work with guidance from a senior backend/AI engineer, and it intentionally avoids asking for everything to be built at once.

The supplied audit is a starting point, not absolute truth. Audits can misclassify code as “DONE” when it is actually a stub, or as missing when it already exists in a different form. Every phase in this document begins with verification against the real repository before any implementation work starts. Wherever this document cannot know an exact file path, function name, or API route without inspecting the code, it says so explicitly: **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

The Development Cycle Used For Every Task

Every single task in every phase — no exceptions — should follow this cycle. Skipping steps (especially INSPECT EXISTING CODE and MANUAL VERIFY) is the most common cause of wasted work in a codebase that already has partially-built features.

1. UNDERSTAND — read the requirement and this document's phase description
2. INSPECT EXISTING CODE — confirm what actually exists before writing anything
3. DESIGN — plan the smallest change that solves the problem
4. IMPLEMENT SMALLEST CHANGE — avoid rewrites and new frameworks
5. RUN TESTS — automated tests for the change
6. MANUAL VERIFY — confirm it works in a real, running system
7. FIX — address anything the tests or manual check reveal
8. DOCUMENT — record what changed and why
9. MOVE TO NEXT TASK

Ground rules for every phase: Do not recommend large rewrites. Do not rewrite working modules merely for style. Do not introduce unnecessary frameworks. Do not duplicate existing functionality. Do not implement optional features before core functionality is stable.

How Each Phase Is Structured

To make this plan usable during real implementation work, every phase (2 through 11) follows the same 20-part structure: Objective, Current Problem, Why This Phase Is Needed, Dependencies, Files/Modules Likely Involved, Step-by-Step Tasks, Expected Code Changes, Database Changes, API Changes, Worker/Background-Job Changes, AI/LLM Changes, Security Considerations, Error-Handling Requirements, Testing Strategy, Manual Verification Steps, Definition of Done, Common Mistakes, Rollback/Recovery Approach, What Should NOT Be Changed, and What The Next Phase Depends On.

No source code is included in this document. This is a task plan and problem-solving roadmap — implementation code should be written directly in the repository, following the steps described here.

PHASE 0 — Repository and System Verification

Objective: Before implementing anything, confirm what is actually true about the current repository. The audit's classifications (DONE / STUB / MISSING) must be independently verified, because incorrect classifications are a common source of wasted implementation effort.

This phase is verification only. Do not implement features during this phase.

Verification Checklist

Work through each area below and record a real status. Use these five statuses consistently across this entire document:

- **VERIFIED COMPLETE** — inspected the code directly and confirmed it works as intended.
- **PARTIALLY COMPLETE** — some of the logic exists, but it is incomplete, unwired, or untested.
- **BROKEN** — code exists but does not work correctly when exercised.
- **MISSING** — no meaningful implementation exists.
- **NOT VERIFIED** — not yet checked; treat as unknown, not as “probably fine.”

System areas to verify:

- environment configuration (.env) loading
- PostgreSQL connectivity and schema state
- Redis connectivity
- Alembic migration status (applied vs. pending)
- FastAPI application startup (no import/config errors)
- existing API routes (which exist, which respond correctly)
- ARQ worker startup and job processing
- LangGraph workflow definition and graph wiring
- specialized agents (which are implemented vs. placeholders)
- RAG components (embeddings, vector store, retrieval)
- LLM provider factory (which providers are registered)
- tool registry (which tools are implemented vs. declared only)
- existing automated test suite (does it run, does it pass)
- Docker build and container startup

Reported Items to Classify

For each item reported in the audit, record the real status found during inspection, plus a short note on what was actually found:

Reported Item	Real Status (fill in)	Notes
process_document_message worker task	NOT VERIFIED	
ingest_knowledge_source worker task	NOT VERIFIED	

Reported Item	Real Status (fill in)	Notes
sync_lead_to_crm worker task	NOT VERIFIED	
OllamaLLMProvider	NOT VERIFIED	
HuggingFaceLLMProvider	NOT VERIFIED	
Stale workflow comments	NOT VERIFIED	
Empty MCP server directory	NOT VERIFIED	
Conversation history loading	NOT VERIFIED	
Knowledge source upload API	NOT VERIFIED	
Workspace/Admin API	NOT VERIFIED	

Definition of Done for Phase 0

- Every system area above has been inspected directly in the repository, not assumed from documentation.
- Every reported item has a real status recorded, replacing the audit's original classification where it differs.
- No implementation work has started yet.
- The team agrees on the Phase 1 starting point based on what was actually found.

PHASE 1 — Infrastructure Verification

Objective: Confirm that the infrastructure every later phase depends on is actually running and reachable, before any application-level work begins.

Exact Checks

PostgreSQL

Confirm the database is running and reachable using the application's configured connection string. Confirm the expected schema/tables exist. Confirm the pgvector extension is installed and enabled, since RAG (Phase 3) depends on it.

Redis

Confirm Redis is running and reachable using the configured connection string. Redis backs both ARQ job queuing and any caching; without it, background jobs (document processing, ingestion, CRM sync) cannot run.

Alembic

Confirm the current migration state (alembic current) matches the latest migration in the repository (alembic heads). Apply any pending migrations in a controlled, reviewed way before proceeding.

FastAPI

Confirm the application starts cleanly with no missing environment variables or import errors, and that a basic health-check route (if one exists) responds successfully.

ARQ

Confirm the ARQ worker process starts, connects to Redis, and picks up a trivially small test job successfully.

Environment variables

Confirm all required environment variables are set: database URL, Redis URL, OpenAI API key (if used), WhatsApp credentials (for real-channel testing), and any other secrets the application expects at startup.

OpenAI connectivity

If OpenAI is configured as the active LLM/embeddings provider, confirm a simple authenticated request succeeds before relying on it in later phases.

Mandatory vs. Optional Services

Service	Mandatory?	Why
PostgreSQL	Mandatory	Core data store for all conversation, lead, and knowledge data.
Redis	Mandatory	Required for ARQ background job processing used across nearly every phase.
Alembic migrations applied	Mandatory	Schema must match code expectations before any phase runs.
OpenAI API key	Mandatory unless a local provider is fully configured	Required for the default LLM/embeddings provider path.
Piper TTS model	Optional	Only required for voice-response features; not a blocker for text-based phases in this roadmap.
Meta WhatsApp credentials	Mandatory for real-channel testing	Required to send/receive real WhatsApp messages; not required for unit/integration tests that mock the channel.
ngrok	Optional / development only	Only needed to expose a local webhook endpoint to Meta for local testing; not used in production deployments.

Definition of Done for Phase 1

- PostgreSQL, Redis, and Alembic are all confirmed working with evidence (command output, logs), not assumption.
- FastAPI and ARQ both start cleanly with the real environment configuration.
- OpenAI connectivity (or the chosen provider) is confirmed with a real successful request.
- Any missing mandatory piece (e.g., Redis not running) is fixed before Phase 2 begins.

PHASE 2

Conversation History Loading

1. Objective

Ensure every incoming WhatsApp message is processed with the real, recent conversation history instead of an empty context, so the agent can understand follow-up questions and multi-turn intent.

2. Current Problem

The audit reports that `run_agent` (or the equivalent workflow entry point) passes `conversation_history = []` instead of loading prior messages for the conversation. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** If confirmed, this means the Supervisor and specialized agents only ever see the single latest message, with no memory of anything said earlier in the conversation.

3. Why This Phase Is Needed

Almost every downstream capability described in the platform (intent detection, lead qualification, RAG follow-up questions, human handoff) depends on the agent understanding what was already discussed. Without history, a customer saying “what about the second option?” cannot be understood. This is why it is placed before RAG ingestion and other feature work: those features will appear broken or low-quality if the underlying context pipeline is not fixed first.

4. Dependencies

- PHASE 0 verification must confirm whether `conversation_history` is really empty, or already partially implemented (it may be `PARTIALLY COMPLETE` rather than `MISSING`).
- PHASE 1 infrastructure verification (PostgreSQL reachable) must pass, since this task reads from the `conversation/message` tables.

5. Files / Modules Likely Involved

- `Worker/agent` entry point that constructs `AgentState` and calls the `LangGraph` workflow (commonly named `run_agent` or similar). **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- `Message/Conversation` persistence layer (SQLAlchemy models and any repository/service functions used to query messages). **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- `AgentState` / workflow state definition used by `LangGraph`. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Confirm the exact signature of `AgentState` and how `conversation_history` is expected to be structured (role, content, timestamp, message type).
2. Identify the existing database model(s) for conversations and messages, and confirm there is already a way to query “last N messages for a conversation, ordered by time.”
3. Write a small, isolated query function that fetches the most recent N messages for a given conversation ID, ordered oldest-to-newest, excluding the message currently being processed.

4. Convert each retrieved message row into the message format the agent/LangGraph layer expects (e.g., role mapped to user/assistant, content as text).
5. Wire this function into the path that builds AgentState, replacing the hardcoded empty list.
6. Add a configurable limit (see below) rather than hardcoding the number of messages loaded.
7. Add logging (structlog) around the history load: conversation ID, number of messages loaded, and load time, so this step is observable in production.

7. Expected Code Changes

- A new or updated query function to load recent messages for a conversation.
- A mapping/adaptor function that converts persisted message rows into the agent's expected message format.
- The call site that builds AgentState updated to use the loaded history instead of an empty list.

8. Database Changes

- None expected if the conversations/messages tables already exist and are populated by the webhook flow.
VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.
- If messages are not currently linked to a conversation_id with a queryable index, a migration to add an index on (conversation_id, created_at) may be required for query performance.

9. API Changes

- None. This phase is internal to the agent execution path and does not add or change public API endpoints.

10. Worker / Background-Job Changes

- The ARQ worker task that invokes the agent workflow must be updated to call the new history-loading function before constructing AgentState.

11. AI / LLM Changes

- AgentState.conversation_history will now be populated, which changes prompt construction. Review the Supervisor/agent prompts to confirm they already expect a history list and format it correctly.
- Consider a maximum token budget for history text so long conversations do not silently overflow the model's context window.

12. Security Considerations

- Ensure the history query is always scoped to the correct workspace/conversation — never load messages across conversations or workspaces.
- Do not log full message content at INFO level if messages may contain PII; log counts and IDs, and keep content logging at DEBUG level behind a flag.

13. Error-Handling Requirements

- If the history query fails (e.g., database timeout), the agent should still proceed with an empty history rather than failing the whole message — log a warning so the degraded behavior is visible.

- Handle the case where a conversation has zero prior messages (first-ever message) as a normal, expected case, not an error.

14. Testing Strategy

- Unit test: given a conversation with N stored messages, the loader returns the correct ordered subset.
- Unit test: the currently-processing message is not duplicated inside the loaded history.
- Unit test: a conversation with no prior messages returns an empty list without raising.
- Integration test: send two sequential WhatsApp-style messages in the same conversation and assert the second agent run receives the first message in its history.

15. Manual Verification Steps

1. Send a first message (e.g., “I need a website”) and confirm a normal response.
2. Send a follow-up message that only makes sense with context (e.g., “how much will that cost?”) and confirm the AI response reflects the earlier topic.
3. Check application logs to confirm the history-load log line shows a non-zero message count on the second message.
4. Inspect the database directly to confirm the loaded messages match what was actually sent.

16. Definition of Done

- conversation_history is populated from the database for every agent run, not hardcoded to [].
- A configurable message limit is in place (see design notes).
- Unit and integration tests above are passing.
- Manual multi-turn test in item 14 above demonstrates contextual understanding.
- Logging shows history load counts in a real run.

17. Common Mistakes

- Loading messages in the wrong order (newest-first) and feeding that directly to the LLM without reversing it — this confuses the model.
- Accidentally including the current incoming message twice (once from the DB query, once as the new message being processed).
- Loading history without a limit, which can blow up token usage and cost on long-running conversations.
- Mixing messages from different conversations or workspaces due to a missing filter.

18. Rollback / Recovery Approach

This change is isolated to one function and one call site. If it causes regressions, the history-loading call can be reverted to pass an empty list while the query function remains in place (feature-flag or single-line revert), with no database rollback required.

19. What Should NOT Be Changed

- Do not restructure the AgentState schema beyond adding/using the conversation_history field it already defines. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

- Do not change the LangGraph graph topology (nodes/edges) as part of this task — this is a data-wiring fix, not a workflow redesign.

20. What The Next Phase Depends On

PHASE 3 (RAG ingestion) and later agent-quality work assume the agent already has real conversational context; verifying this phase first avoids debugging RAG relevance issues that are actually context issues.

PHASE 3

RAG Knowledge Ingestion

1. Objective

Implement (or complete) the `ingest_knowledge_source` worker task so that documents and text added to the knowledge base are reliably converted into searchable embeddings.

2. Current Problem

The audit lists `ingest_knowledge_source` as a reported remaining item. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** The likely current state is that a `KnowledgeSource` record can be created, but the pipeline that extracts, chunks, embeds, and stores its content (`DocumentChunk` rows with `pgvector` embeddings) is missing, a stub, or incomplete.

3. Why This Phase Is Needed

The Knowledge Agent and RAG-grounded answers described in the platform architecture cannot function without a working ingestion pipeline — there is nothing to retrieve. This is placed before the Knowledge Source Upload API (Phase 4) because the ingestion worker is the core logic the API will enqueue; building the API first would have nothing working to call.

4. Dependencies

- PHASE 1 must confirm PostgreSQL + `pgvector` extension is enabled and Redis/ARQ are operational, since ingestion runs as a background job.
- PHASE 2 does not block this phase technically, but should be completed first per the required order.

5. Files / Modules Likely Involved

- ARQ worker task module containing (or intended to contain) `ingest_knowledge_source`. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- `KnowledgeSource` and `DocumentChunk` SQLAlchemy models. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Any existing text-extraction utilities (PDF, OCR via PaddleOCR) and the embeddings provider (Sentence Transformers / OpenAI embeddings). **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Confirm the `KnowledgeSource` model's fields: source type (PDF/text/URL), status field, and any existing status values (e.g., `PENDING`, `PROCESSING`, `COMPLETED`, `FAILED`).
2. Confirm whether text-extraction utilities already exist for PDF and plain text, and whether an embeddings provider function is already implemented and tested elsewhere in the codebase.
3. Implement the ingestion task as a sequence of small, independently testable steps: load source → extract text → clean/normalize → chunk → embed → persist `DocumentChunk` rows → mark source `COMPLETED`.
4. Add explicit handling for an empty or unreadable document: mark the source `FAILED` with a clear reason rather than leaving it stuck in `PROCESSING`.

5. Add duplicate-ingestion protection: re-running ingestion for the same source should not create duplicate chunks (e.g., delete-and-replace or a source-version check).
6. Add basic retry behavior for transient failures (e.g., embedding API timeout) distinct from permanent failures (e.g., corrupt file), so only the former is retried.
7. Emit structured logs at each pipeline step with the source ID, so a stuck or failed ingestion can be diagnosed from logs alone.

7. Expected Code Changes

- ingest_knowledge_source worker task implementation (or completion of the stub).
- Chunking utility (text splitting with reasonable overlap).
- Embedding-generation call wired to the existing embeddings provider.
- Persistence logic that writes DocumentChunk rows with their vector embeddings.

8. Database Changes

- None expected to the schema if KnowledgeSource/DocumentChunk tables and the pgvector column already exist. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- If a status/error-message column does not exist on KnowledgeSource, an Alembic migration to add it is recommended so failures are visible without reading logs.

9. API Changes

- None in this phase — the API surface is addressed in Phase 4.

10. Worker / Background-Job Changes

- New or completed ARQ task: ingest_knowledge_source.
- Task should be idempotent: running it twice for the same source should not double-insert chunks.

11. AI / LLM Changes

- Embeddings are generated using the existing configured provider (Sentence Transformers locally, or an OpenAI embeddings call). **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- No change to the LangGraph workflow itself in this phase — this phase only makes retrieval data available.

12. Security Considerations

- Validate that extracted text does not get passed anywhere as executable content; treat all extracted text as untrusted data.
- Ensure ingestion is scoped to the correct workspace so one business's documents cannot leak into another workspace's retrieval results.

13. Error-Handling Requirements

- Extraction failure (corrupt PDF, unsupported format) → mark source FAILED with a specific reason.
- Embedding failure (API error/timeout) → retry with backoff a bounded number of times, then mark FAILED.
- Partial failure (some chunks embed, others fail) → decide and document whether the source is marked PARTIAL/FAILED or retried as a whole; do not silently leave it COMPLETED with missing chunks.

14. Testing Strategy

- Unit test: chunking function produces expected chunk counts/boundaries for sample text.
- Unit test: empty document input results in a FAILED status, not a crash.
- Integration test: ingesting a small real PDF results in DocumentChunk rows with non-null embeddings.
- Integration test: re-running ingestion on the same source does not duplicate chunks.
- Integration test: a similarity search against the ingested chunks returns the expected chunk for a known query.

15. Manual Verification Steps

1. Create a KnowledgeSource pointing to a short, real PDF or text sample.
2. Trigger ingestion (directly via the worker task, since the API is not built until Phase 4) and watch logs for each pipeline step.
3. Query the database to confirm DocumentChunk rows exist with embeddings and correct source linkage.
4. Run a manual similarity query for a known phrase from the document and confirm relevant chunks return.

16. Definition of Done

- ingest_knowledge_source reliably processes PDF and raw text sources end-to-end.
- Failures are captured with a status and reason, not silent or stuck states.
- Duplicate ingestion does not create duplicate chunks.
- Tests above pass, and the manual verification steps succeed against a real sample document.

17. Common Mistakes

- Chunking too large (poor retrieval precision) or too small (loses context) without testing retrieval quality on real content.
- Not normalizing whitespace/encoding from PDF extraction, leading to noisy embeddings.
- Leaving a source stuck in PROCESSING forever when an unhandled exception occurs mid-pipeline.
- Re-ingesting a source and leaving old chunks in place alongside new ones (silent duplication).

18. Rollback / Recovery Approach

Because ingestion writes new rows rather than modifying existing platform behavior, rollback is low-risk: disable the worker task (stop consuming the job) and, if needed, delete DocumentChunk rows for affected sources. No other platform functionality depends on this phase yet.

19. What Should NOT Be Changed

- Do not change the pgvector column type or embedding dimensionality without confirming it matches the embeddings provider currently configured. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Do not replace the existing embeddings provider (Sentence Transformers/OpenAI) with a new library as part of this task — that is out of scope and would be an unnecessary framework change.

20. What The Next Phase Depends On

PHASE 4 (Knowledge Source Upload API) enqueues this exact worker task, so it must be functioning correctly and idempotently before the API is built on top of it.

PHASE 4

Knowledge Source Upload API

1. Objective

Expose an authenticated, workspace-scoped API endpoint that lets a business owner submit a new knowledge source (document, text, or URL) and track its ingestion status.

2. Current Problem

The audit lists the knowledge source upload API as a reported remaining item. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** Even once Phase 3's ingestion worker works, there may be no supported way for a user to actually create a KnowledgeSource and trigger it through the API/dashboard.

3. Why This Phase Is Needed

This is the first user-facing entry point into the RAG pipeline. It is sequenced immediately after Phase 3 so the API has a working, tested worker task to enqueue — building the API earlier would risk exposing a broken pipeline to users.

4. Dependencies

- PHASE 3 must be complete and tested: ingest_knowledge_source must reliably process a source end-to-end.
- Existing authentication/authorization mechanism must be confirmed and reused, not reinvented.

5. Files / Modules Likely Involved

- FastAPI router module for knowledge endpoints. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Pydantic request/response schemas for KnowledgeSource. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Existing auth/dependency-injection utilities (current user, current workspace). **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- ARQ client/enqueue utility used elsewhere in the codebase to submit worker jobs. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Confirm the existing authentication dependency pattern used by other endpoints (e.g., a FastAPI dependency that resolves the current user and workspace) and reuse it exactly.
2. Define request schemas for the supported source types described in this document: PDF upload, raw text, and URL.
3. Implement POST /knowledge/sources (or the equivalent route already scaffolded, if one exists) to: validate input → create a KnowledgeSource row scoped to the caller's workspace → enqueue the ingest_knowledge_source ARQ job → return the created source with its initial status.

4. Implement a GET endpoint (or reuse an existing pattern) to fetch a source's current status, so the frontend can poll for COMPLETED/FAILED.
5. Add file-size and file-type validation before accepting an upload.
6. Add workspace isolation checks so a user from Workspace A cannot create, view, or trigger ingestion for a source under Workspace B.

7. Expected Code Changes

- New Pydantic schemas: KnowledgeSourceCreate, KnowledgeSourceResponse (or equivalent).
- New/updated FastAPI route handlers for creating and viewing knowledge sources.
- Wiring from the route handler to the ARQ enqueue call for ingest_knowledge_source.

8. Database Changes

- None expected beyond what Phase 3 already required, assuming KnowledgeSource already models the needed fields. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

9. API Changes

- POST /knowledge/sources — create a knowledge source and enqueue ingestion.
- GET /knowledge/sources/{id} — retrieve a single source's status and metadata.
- GET /knowledge/sources — list sources for the current workspace (recommended for dashboard use).

10. Worker / Background-Job Changes

- None beyond Phase 3 — this phase only adds the enqueue call from the API layer.

11. AI / LLM Changes

- None. This phase is API/plumbing only.

12. Security Considerations

- Enforce authentication on every endpoint; reject unauthenticated requests.
- Enforce workspace-level authorization on every read/write — always filter by the authenticated user's workspace, never trust a workspace ID supplied only by the client.
- Enforce a maximum upload file size and an allow-list of accepted file types/MIME types.
- Sanitize or reject suspicious URLs submitted for URL-based ingestion (see SSRF note in Phase 11).

13. Error-Handling Requirements

- Return clear 4xx responses for invalid input (bad file type, oversized file, missing fields).
- Return 404 (not 500) when a source ID does not exist or does not belong to the caller's workspace.
- If enqueueing the ARQ job fails (e.g., Redis unavailable), return a 503-style error rather than silently creating a source that will never be processed.

14. Testing Strategy

- API test: authenticated user can create a knowledge source and receives a PENDING/PROCESSING status.
- API test: unauthenticated requests are rejected.
- API test: a user cannot view or act on a source belonging to a different workspace.
- API test: oversized or invalid file types are rejected with a clear error.
- Integration test: creating a source via the API results in the ARQ job being enqueued and, eventually, the source status becoming COMPLETED.

15. Manual Verification Steps

1. Use an API client (e.g., curl or an API testing tool) to create a knowledge source as an authenticated user.
2. Confirm the response includes a source ID and initial status.
3. Poll the GET endpoint until status becomes COMPLETED and confirm chunk data exists in the database.
4. Attempt the same request without authentication and confirm it is rejected.

16. Definition of Done

- Endpoints exist, are authenticated, and are workspace-isolated.
- Valid submissions result in successful ingestion end-to-end through the API.
- Invalid submissions are rejected with clear, correct HTTP status codes.
- API tests above pass.

17. Common Mistakes

- Trusting a workspace_id field from the request body instead of deriving it from the authenticated session.
- Skipping file-size limits, allowing very large uploads to exhaust worker memory.
- Returning 500 errors for expected validation failures instead of proper 4xx responses.
- Not returning enough status information for the frontend to show ingestion progress.

18. Rollback / Recovery Approach

This phase only adds new routes; it does not modify existing endpoints. Rollback is a simple route removal/disable if problems are found, with no impact on already-ingested data.

19. What Should NOT Be Changed

- Do not modify unrelated existing API routers while adding this one.
- Do not change the authentication mechanism itself — reuse what already exists.

20. What The Next Phase Depends On

PHASE 5 (WhatsApp document processing) reuses the same extraction/ingestion building blocks confirmed working here, and PHASE 6 (Workspace/Admin API) follows the same authentication and workspace-isolation pattern established in this phase.

PHASE 5

WhatsApp Document Processing

1. Objective

Implement `process_document_message` so that PDFs, Word documents, and CSV files sent by customers directly over WhatsApp are downloaded, text-extracted (with OCR fallback for scanned content), and fed into the same agent pipeline as text messages.

2. Current Problem

The audit lists `process_document_message` as a reported remaining item. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** The likely current state is that the webhook can receive a document message and enqueue a job, but the worker logic to download the media, detect its type, extract content, and hand it to `run_agent` is missing or incomplete.

3. Why This Phase Is Needed

This phase depends directly on extraction utilities that should already be proven out during Phase 3 (PDF text extraction) and Phase 4 (validation patterns), which is why it is sequenced after them rather than before. Building document handling before the extraction pipeline is stable would mean re-solving the same problems twice.

4. Dependencies

- PHASE 3 ingestion pipeline extraction utilities should already exist and be tested for PDFs.
- PHASE 2 conversation history should be working, since the extracted document content will be handled as part of a conversation.
- Meta WhatsApp media-download credentials must be configured (see Phase 1) to fetch the actual file.

5. Files / Modules Likely Involved

- ARQ worker task module containing (or intended to contain) `process_document_message`. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- WhatsApp webhook handler that currently receives document message events. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- PaddleOCR integration point for scanned/image-based content. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Any existing media-download client for the WhatsApp Cloud API. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Confirm how the webhook currently identifies an incoming document message and what payload (media ID, MIME type, filename) is already available to the worker.
2. Implement/complete the media download step using the existing WhatsApp media client, saving the file to a temporary, workspace-scoped location.

3. Implement document-type detection based on MIME type/extension (PDF, DOCX, CSV) and route to the correct extraction path for each.
4. For PDFs, first attempt direct text extraction; if the extracted text is empty or below a reasonable length threshold, treat the PDF as scanned and fall back to PaddleOCR.
5. For Word and CSV files, implement straightforward text/content extraction appropriate to each format.
6. Persist the extracted content in a way that mirrors how conversation messages are stored, so it can be included in conversation history and passed into `run_agent` like a text message.
7. Create the `AgentRun` and call `run_agent` with the extracted content, then send the resulting response back to the customer over WhatsApp exactly as a normal text reply would be sent.
8. Add cleanup of any temporary downloaded files after processing completes or fails.

7. Expected Code Changes

- `process_document_message` worker task implementation.
- Document-type detection and per-format extraction functions (PDF, DOCX, CSV).
- OCR fallback integration for scanned PDFs/images via PaddleOCR.
- Adapter that turns extracted document content into a conversation message the agent can consume.

8. Database Changes

- Likely none if the Message model already supports a document/attachment message type; confirm this before adding new columns. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- If there is no way to record that a message originated from a document (vs. plain text), consider a minimal column/enum addition via Alembic migration.

9. API Changes

- None. This phase is entirely webhook/worker driven, not a public API.

10. Worker / Background-Job Changes

- New or completed ARQ task: `process_document_message`, enqueued by the existing webhook handler for document message events.

11. AI / LLM Changes

- The agent receives extracted document text as if it were a message from the user; no change to the LangGraph graph itself should be required.
- Very large extracted documents should be truncated or summarized before being passed to the LLM to avoid exceeding context limits — define a sensible maximum length.

12. Security Considerations

- Validate file size and type before download/processing to avoid resource exhaustion from oversized or malicious files.
- Treat all extracted text as untrusted user input; do not interpret it as instructions to the system (see prompt-injection guidance in Phase 11).
- Ensure temporary files are stored and deleted safely, with no cross-workspace access.

13. Error-Handling Requirements

- Unsupported file type → respond to the customer with a clear message that the format is not supported, rather than failing silently.
- OCR/extraction failure → respond gracefully and log the failure with enough detail to diagnose it, without crashing the worker.
- Media download failure (expired link, network error) → retry a bounded number of times before giving up and notifying the customer.

14. Testing Strategy

- Unit test: document-type detection correctly routes PDF/DOCX/CSV inputs.
- Unit test: a text-based PDF returns extracted text without invoking OCR.
- Unit test: a scanned/empty-text PDF triggers the OCR fallback path.
- Integration test: an end-to-end run from a sample document to a generated agent response.
- Test with at least one intentionally malformed file to confirm graceful failure.

15. Manual Verification Steps

1. Using a configured WhatsApp test number, send a text-based PDF and confirm a relevant AI response.
2. Send a scanned (image-only) PDF and confirm OCR is used and a reasonable response is produced.
3. Send an unsupported file type and confirm a clear, friendly rejection message is returned.
4. Check logs to confirm each pipeline stage (download, detect, extract/OCR, agent run, response sent) is visible and traceable by a correlation/conversation ID.

16. Definition of Done

- PDF (text and scanned), Word, and CSV documents sent over WhatsApp are processed end-to-end.
- Unsupported formats and failures are handled gracefully with a customer-facing message.
- Tests above pass and manual WhatsApp verification succeeds for at least one file of each supported type.

17. Common Mistakes

- Assuming all PDFs are text-based and skipping the OCR fallback, producing empty responses for scanned documents.
- Not cleaning up temporary downloaded files, leading to disk usage growth over time.
- Passing extremely large extracted text directly into the LLM prompt without truncation.
- Not distinguishing between a permanent failure (bad file) and a transient one (network hiccup) when deciding whether to retry.

18. Rollback / Recovery Approach

If document processing proves unstable, the webhook can be configured to acknowledge document messages with a static “we received your file” response while the worker task is disabled, without affecting text-message handling, which is a separate code path.

19. What Should NOT Be Changed

- Do not modify the text-message handling path while implementing document handling — they should remain separate, independently testable flows.
- Do not introduce a new OCR or document-parsing library when PaddleOCR and existing PDF tooling are already part of the stack.

20. What The Next Phase Depends On

No later phase strictly depends on Phase 5's internals, but its extraction patterns should be reused (not duplicated) if similar needs arise in later phases.

PHASE 6

Workspace / Admin API

1. Objective

Provide authenticated endpoints for a business owner to view and update their workspace's business profile, AI persona, tone, custom instructions, and escalation configuration.

2. Current Problem

The audit lists the workspace/admin API as a reported remaining item. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** The likely current state is that persona/business-profile data may already exist as database fields used internally by the agents, but there is no supported API surface for an authenticated user to read or change them.

3. Why This Phase Is Needed

Every business-customization capability described in the platform (tone, persona, escalation rules) is meaningless without a way for the business owner to configure it. It is sequenced after the knowledge and document phases because those are core conversational capabilities, while workspace configuration is an administrative layer on top of a working core.

4. Dependencies

- The authentication/authorization pattern established in Phase 4 must be reused here.
- The underlying BusinessProfile/workspace configuration model must already exist or be confirmed as missing. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

5. Files / Modules Likely Involved

- FastAPI router module for workspace/admin endpoints. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- BusinessProfile or equivalent SQLAlchemy model and its fields (tone, persona, instructions, escalation rules). **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Existing auth dependencies for current user/workspace. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Confirm the existing data model for business profile/persona configuration and exactly which fields already exist versus need to be added.
2. Define Pydantic schemas for reading and updating the workspace profile, keeping fields aligned with what the agents actually consume (do not invent fields the agents cannot use).
3. Implement GET /workspace/profile to return the current configuration for the authenticated user's workspace.
4. Implement PUT/PATCH /workspace/profile to update allowed fields, with validation (e.g., tone must be one of a defined set of values).

5. Implement a focused endpoint or sub-resource for escalation configuration if this is modeled separately from the general profile.
6. Ensure only users with an appropriate role (e.g., workspace admin/owner) can update configuration, if role distinctions already exist in the system.

7. Expected Code Changes

- New Pydantic schemas for workspace profile read/update.
- New FastAPI route handlers under a workspace/admin router.
- Validation logic for allowed tone/persona values and escalation rule structure.

8. Database Changes

- None expected if BusinessProfile already models the necessary fields; if fields such as escalation rules do not yet exist, an Alembic migration to add them will be required. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

9. API Changes

- GET /workspace/profile — view current business profile and persona configuration.
- PUT /workspace/profile — update allowed profile/persona fields.
- GET/PUT for escalation configuration, if modeled as a separate resource.

10. Worker / Background-Job Changes

- None. This phase is API/CRUD only and does not touch background jobs.

11. AI / LLM Changes

- No workflow change, but updated persona/tone values must be confirmed to actually be read by the Supervisor/agents at prompt-construction time — **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

12. Security Considerations

- Enforce workspace isolation exactly as in Phase 4 — a user can only view/edit their own workspace.
- If role-based permissions exist, restrict write access to admin/owner roles and allow read-only access to other workspace members, if applicable.
- Validate all free-text fields (custom instructions) for length limits to prevent excessively long prompts being injected into every future agent call.

13. Error-Handling Requirements

- Return 422 with clear field-level errors for invalid tone/persona values.
- Return 404 if a workspace profile does not yet exist for a new workspace, or auto-create a sensible default — decide and document which behavior is intended.

14. Testing Strategy

- API test: authenticated user can read their workspace profile.
- API test: authenticated user can update allowed fields and the change persists.

- API test: invalid tone/persona values are rejected.
- API test: a user cannot read or modify another workspace's profile.

15. Manual Verification Steps

1. Update the persona tone via the API (e.g., to “casual”) and then send a WhatsApp test message to confirm the AI's response style changes accordingly.
2. Attempt to access another workspace's profile with a different authenticated user and confirm it is denied.

16. Definition of Done

- Workspace profile can be read and updated through authenticated, workspace-isolated endpoints.
- Updated persona/tone/instructions are confirmed to actually influence agent behavior end-to-end.
- Tests above pass.

17. Common Mistakes

- Adding configuration fields to the API that the agents never actually read, giving a false impression of customization.
- Allowing unrestricted free-text length in custom instructions, inflating token usage on every request.
- Forgetting role checks and allowing any workspace member to change escalation-critical settings.

18. Rollback / Recovery Approach

New, additive endpoints; disabling the router or reverting the migration (if one was added) fully removes this phase's surface without affecting existing conversation or knowledge data.

19. What Should NOT Be Changed

- Do not change how the Supervisor/agents read persona data unless it is confirmed broken — this phase is about exposing configuration, not redesigning how agents consume it.

20. What The Next Phase Depends On

No later phase strictly depends on this one, but CRM sync (Phase 7) and MCP tool permissions (Phase 8) should follow the same authentication/authorization pattern established here and in Phase 4.

PHASE 7

CRM Synchronization

1. Objective

Implement `sync_lead_to_crm` so that qualified leads captured by the Lead Agent are reliably pushed to an external CRM (starting with a mock provider, then HubSpot) without duplication.

2. Current Problem

The audit lists `sync_lead_to_crm` as a reported remaining item. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** The likely current state is that leads are created in PostgreSQL by the Lead Qualification Agent, but there is no reliable background job that forwards them to an external CRM system.

3. Why This Phase Is Needed

This is sequenced after the core conversational and knowledge phases because CRM sync is an outward integration on top of already-working lead capture; syncing broken or incomplete lead data outward would create confusing, low-quality CRM records.

4. Dependencies

- Lead capture via the Lead Qualification Agent must already be functioning and storing complete lead records. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- PHASE 1 must confirm Redis/ARQ are operational, since this runs as a background job.

5. Files / Modules Likely Involved

- ARQ worker task module containing (or intended to contain) `sync_lead_to_crm`. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Lead SQLAlchemy model, including any existing `crm_synced_at` or similar tracking field. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Any existing CRM provider abstraction/interface. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Confirm the Lead model's current fields, especially whether a sync-status/timestamp field (e.g., `crm_synced_at`) already exists. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
2. Design a small CRM provider interface with at least two implementations: a Mock provider (for tests and local development) and a HubSpot provider (for production use).
3. Implement the Mock provider first and get the full sync flow working end-to-end against it before touching real HubSpot credentials.
4. Implement `sync_lead_to_crm` to: load the lead → check if already synced (idempotency) → call the configured CRM provider → update `crm_synced_at` (and a status/error field) → log the outcome.
5. Implement the HubSpot provider using HubSpot's supported API for contact/deal creation, mapping only the lead fields that are already captured (do not invent fields the Lead Agent does not collect).

6. Add retry-with-backoff for transient CRM API failures, and a clear FAILED state for permanent failures (e.g., invalid credentials, rejected payload).
7. Add an audit log entry for every sync attempt (success or failure) for traceability.

7. Expected Code Changes

- `sync_lead_to_crm` worker task implementation.
- CRM provider interface plus Mock and HubSpot implementations.
- Idempotency check before each sync attempt.

8. Database Changes

- If not already present, an Alembic migration to add `crm_synced_at`, `crm_sync_status`, and/or `crm_sync_error` fields to the Lead table. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

9. API Changes

- None required for the core sync mechanism; an optional manual “resync lead” admin endpoint may be considered later but is not part of this phase’s scope.

10. Worker / Background-Job Changes

- New ARQ task: `sync_lead_to_crm`, triggered when a lead is created/qualified or on a scheduled retry for previously failed syncs.

11. AI / LLM Changes

- None. This phase is integration/plumbing and does not change agent or LLM behavior.

12. Security Considerations

- Store CRM API credentials using the existing secret-management approach; never hardcode them.
- Ensure only lead fields the business is entitled to send are transmitted — do not forward unrelated internal metadata to the external CRM.
- Log sync attempts without exposing full credential values in logs.

13. Error-Handling Requirements

- Transient errors (timeouts, rate limits) → retry with backoff, bounded attempts.
- Permanent errors (invalid credentials, malformed payload) → mark FAILED immediately, do not retry indefinitely, and surface the error for an operator to review.
- Never silently drop a lead sync failure — it must be visible in logs and/or an audit table.

14. Testing Strategy

- Unit test: idempotency — a lead already marked synced is not re-sent to the CRM provider.
- Unit test: Mock provider integration produces the expected `crm_synced_at` update.
- Unit test: permanent failure sets a FAILED status without infinite retries.
- Integration test (where feasible): HubSpot provider against a sandbox/test account, if available.

15. Manual Verification Steps

1. Create a test lead and confirm it syncs successfully against the Mock provider, with `crm_synced_at` set.
2. Force a failure (e.g., invalid mock config) and confirm the lead is marked **FAILED** with a logged reason, not left in a silent pending state.
3. If HubSpot credentials are available, confirm one real test lead appears correctly in the HubSpot sandbox/test account.

16. Definition of Done

- Leads sync reliably to the CRM provider exactly once per lead under normal conditions.
- Failures are visible, logged, and bounded (no infinite retry loops).
- Mock provider fully tested; HubSpot provider implemented and manually verified against at least one real test lead.

17. Common Mistakes

- Re-syncing the same lead on every retry of an unrelated job, creating duplicate CRM records.
- Retrying permanent failures (e.g., bad credentials) indefinitely instead of failing fast.
- Hardcoding HubSpot-specific field mappings in a way that makes adding a second CRM provider difficult later.

18. Rollback / Recovery Approach

Disabling the sync worker task stops outward CRM calls immediately without affecting lead data in PostgreSQL, which remains the source of truth. No destructive database changes are made by this phase.

19. What Should NOT Be Changed

- Do not modify the Lead Qualification Agent's extraction logic as part of this phase — this phase only forwards already-captured leads outward.
- Do not build a general-purpose CRM abstraction beyond what is needed for Mock + HubSpot; avoid over-engineering for hypothetical future CRMs.

20. What The Next Phase Depends On

No later phase strictly depends on CRM sync, but the provider-interface pattern used here (Mock + real implementation) is the same pattern recommended for MCP external tools in Phase 8.

PHASE 8

MCP Architecture and External Tool Integration

1. Objective

Establish a clear, hybrid tool architecture that keeps internal/native LangGraph tools for internal operations while introducing MCP (Model Context Protocol) for reusable external integrations, without treating one as a wholesale replacement for the other.

2. Current Problem

The audit notes an empty MCP server directory. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** This suggests MCP was planned but not yet implemented, and no governance/tool-security layer currently distinguishes internal versus external tool calls.

3. Why This Phase Is Needed

MCP is sequenced late because it is additive infrastructure for external integrations (HubSpot, calendar, other SaaS tools), not a prerequisite for the platform's core conversational and RAG functionality. Introducing it before the core is stable would add complexity without corresponding value.

4. Dependencies

- PHASE 7 (CRM synchronization) should already demonstrate a working external-integration pattern; MCP should build on those lessons rather than duplicate them.
- A tool registry/permission concept must exist or be introduced as part of this phase.

5. Files / Modules Likely Involved

- The currently empty MCP server directory referenced in the audit. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Existing native tool implementations and however they are currently registered with LangGraph. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Any existing tool-permission or governance code. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Document the hybrid architecture decision explicitly in the codebase (a short design note): native tools remain the default for internal database operations, business logic, authentication, authorization, and consent; MCP is introduced specifically for reusable external/SaaS integrations.
2. Design a governance/security layer that sits between LangGraph tool calls and their execution, regardless of whether the tool is native or MCP-based, so permission checks are consistent.
3. Define a minimal tool registry: tool name, type (native/MCP), required permissions, and owning module.
4. Implement MCP tool discovery for a first, low-risk external integration candidate (e.g., a calendar or an already-partially-integrated CRM), validating tool definitions before they are made available to agents.

5. Add explicit checks against the known MCP risk classes: tool poisoning (malicious tool definitions), tool shadowing (a malicious tool masquerading as a trusted one), prompt injection through tool results, and excessive permissions.
6. Ensure credentials used by MCP tools are isolated per integration and never shared with unrelated tools or workspaces.
7. Add audit logging for every tool call (native and MCP) including tool name, workspace, and outcome.
8. Benchmark end-to-end latency (p50/p95/p99) for at least one native tool and one MCP tool under similar conditions before drawing any performance conclusions.

7. Expected Code Changes

- A tool registry module covering both native and MCP tools.
- A governance/permission-check layer invoked before any tool executes.
- An MCP server/client integration for at least one pilot external tool.
- Audit logging hooks around tool execution.

8. Database Changes

- Optional: an `audit_logs` table for tool execution history, if one does not already exist. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

9. API Changes

- None required directly, unless tool permissions are exposed via the Workspace/Admin API from Phase 6 for configuration purposes.

10. Worker / Background-Job Changes

- None directly; tool calls happen within the LangGraph workflow invoked by existing worker tasks.

11. AI / LLM Changes

- The Supervisor/agents gain access to MCP-based tools alongside native tools, mediated through the same governance layer.
- Tool results returned from MCP servers must be treated as untrusted content and should not be interpreted as system instructions (prompt-injection defense).

12. Security Considerations

- Apply least-privilege permissions per tool — a tool should only be able to do what it explicitly needs to do.
- Validate MCP tool definitions before registering them (schema validation, source trust).
- Isolate credentials per external integration; do not share one broad credential across multiple tools.
- Defend against tool shadowing by pinning trusted tool sources/identities rather than matching on name alone.
- Treat all MCP tool output as untrusted data when it re-enters the LLM context.

13. Error-Handling Requirements

- A failing or unreachable MCP server should degrade gracefully (the agent should be able to say it cannot complete that action) rather than crashing the conversation.
- Tool-call timeouts must be enforced so a slow external tool cannot stall the whole agent run.

14. Testing Strategy

- Unit test: governance layer rejects a tool call that exceeds its permitted scope.
- Unit test: malformed/untrusted tool definitions are rejected at registration time.
- Integration test: a pilot MCP tool can be discovered, invoked, and its result correctly returned to the agent.
- Security test: simulate a tool result containing an injected instruction and confirm the agent does not treat it as a command.

15. Manual Verification Steps

1. Trigger a conversation flow that uses a native tool and confirm the governance layer logs the call.
2. Trigger a conversation flow that uses the pilot MCP tool and confirm the same governance/audit logging occurs.
3. Record p50/p95/p99 latency for both calls and compare, without assuming in advance which is faster.

16. Definition of Done

- A documented hybrid architecture exists and is followed: native tools for internal operations, MCP for external integrations.
- A governance layer enforces permissions consistently for both tool types.
- At least one MCP tool is implemented, tested, and audited end-to-end.
- Latency has been measured, not assumed.

17. Common Mistakes

- Assuming MCP is inherently slower or native tools are inherently sub-millisecond without measuring.
- Granting an MCP tool broad database or internal-system access instead of a narrow, purpose-specific credential.
- Skipping validation of tool definitions, allowing a malicious or malformed tool to be registered.
- Feeding raw MCP tool output back into the LLM prompt without treating it as untrusted content.

18. Rollback / Recovery Approach

Because MCP is additive, the pilot integration and its registry entry can be disabled without affecting native tool behavior or any other phase's functionality.

19. What Should NOT Be Changed

- Do not migrate existing, working native tools to MCP as part of this phase merely for consistency — only introduce MCP where external/reusable integration is the actual need.
- Do not remove or bypass the governance layer for “trusted” tools — all tool calls should pass through it.

20. What The Next Phase Depends On

No later phase strictly depends on MCP; Phases 9 and 10 (optional LLM providers) are independent of the tool architecture.

PHASE 9

Ollama Provider (Optional)

1. Objective

Add OllamaLLMProvider as an additional, optional LLM provider so the platform can run against a locally hosted model when that is desirable (cost control, data locality, offline development), without making it a requirement for core functionality.

2. Current Problem

The audit lists OllamaLLMProvider as a reported remaining item. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** The provider factory pattern for LLMs likely already supports OpenAI; Ollama support is additional, not a fix to broken functionality.

3. Why This Phase Is Needed

This is explicitly optional and is placed late in the roadmap so that core conversational, RAG, and integration functionality is stable and well-tested against the primary provider (OpenAI) before introducing a second provider with different capabilities and limitations.

4. Dependencies

- The existing LLM provider interface/factory must be understood first, since Ollama support should implement that same interface rather than introduce a parallel one.
- Core agent workflow (Phases 2–6) should already be stable, since this phase changes which model powers it, not what it does.

5. Files / Modules Likely Involved

- Existing LLM provider factory/interface module. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Configuration/settings module where provider selection is configured. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Confirm the existing provider interface's required methods (e.g., chat/completion call, streaming support if any, structured-output handling) before writing new code.
2. Implement OllamaLLMProvider against Ollama's /api/chat endpoint, matching the existing provider interface exactly.
3. Handle JSON/structured responses carefully: confirm which locally available models support reliable structured output and document which do not.
4. Handle tool calling as model-dependent: only enable tool-calling code paths for Ollama models that are confirmed to support it; fail clearly (not silently) for models that do not.
5. Add configuration to select Ollama as the active provider without code changes (environment-based provider selection, consistent with the existing factory pattern).

6. Add clear failure handling for the case where the local Ollama server is not running or unreachable.

7. Expected Code Changes

- OllamaLLMProvider implementation satisfying the existing provider interface.

8. Database Changes

- None.

9. API Changes

- None. Provider selection is a configuration concern, not an API surface change.

10. Worker / Background-Job Changes

- None directly; the worker/agent layer should be unaffected since it depends on the provider interface, not a specific provider.

11. AI / LLM Changes

- Response quality, latency, and structured-output reliability will differ from OpenAI and should be evaluated before recommending Ollama for any production workspace.
- Tool-calling support varies by local model; this must be explicitly tested per model, not assumed.

12. Security Considerations

- If Ollama runs on a shared or networked host, ensure it is not exposed without access control.
- Do not send workspace data to any remote endpoint when using a local model — confirm the client is actually pointed at a local/private Ollama instance.

13. Error-Handling Requirements

- Local server unreachable → fail fast with a clear error rather than a long timeout on every request.
- Unsupported structured-output or tool-calling request for the configured model → return a clear, specific error rather than a malformed response.

14. Testing Strategy

- Unit test: OllamaLLMProvider conforms to the same interface contract as the existing OpenAI provider (can be swapped without changing calling code).
- Integration test (requires local Ollama): a basic chat completion succeeds.
- Integration test: structured-output request behavior is verified for at least one supported model.

15. Manual Verification Steps

1. Run a local Ollama server with a supported model and switch the platform's provider configuration to Ollama.
2. Send a simple test conversation and confirm a coherent response.
3. Attempt a tool-calling scenario and confirm either correct behavior or a clear, documented failure for unsupported models.

16. Definition of Done

- OllamaLLMProvider implements the existing provider interface without special-casing calling code.
- Structured-output and tool-calling support/limitations are documented per tested model.
- Core platform tests still pass with OpenAI as the default provider (Ollama does not become a hidden requirement).

17. Common Mistakes

- Making Ollama a hard dependency for local development or testing, blocking contributors without a local model set up.
- Assuming all Ollama-hosted models support tool calling and structured output identically to OpenAI.
- Building a separate code path in the agent layer specifically for Ollama instead of relying on the shared provider interface.

18. Rollback / Recovery Approach

Provider selection remains configuration-driven; reverting to OpenAI as the active provider requires only a configuration change, with no impact on other phases.

19. What Should NOT Be Changed

- Do not alter the existing OpenAI provider or the shared provider interface contract to accommodate Ollama-specific quirks — handle differences inside the Ollama provider implementation.

20. What The Next Phase Depends On

PHASE 10 follows the same optional-provider pattern established here.

PHASE 10

HuggingFace Provider (Optional)

1. Objective

Add HuggingFaceLLMProvider as a second optional provider option, for teams that want to use HuggingFace-hosted or self-hosted inference endpoints, following the same non-mandatory approach as the Ollama provider.

2. Current Problem

The audit lists HuggingFaceLLMProvider as a reported remaining item. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** As with Ollama, this is additive provider support, not a fix to broken core functionality.

3. Why This Phase Is Needed

Sequenced last among provider work, after Ollama, because it is the least likely to be immediately needed and because validating the provider-interface pattern once (in Phase 9) makes this implementation faster and lower-risk.

4. Dependencies

- The same provider interface confirmed/used in Phase 9 must be reused here.
- A decision on which HuggingFace integration path to use (hosted Inference API vs. a self-hosted endpoint) must be made based on the project's actual needs. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

5. Files / Modules Likely Involved

- Existing LLM provider factory/interface module (same as Phase 9). **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**
- Configuration/settings module for provider and credential selection. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Confirm which HuggingFace access pattern is actually intended: the hosted Inference API (requiring an API token) or a self-hosted/Text Generation Inference-style endpoint.
2. Implement HuggingFaceLLMProvider against the chosen access pattern, matching the existing provider interface used by OpenAI and Ollama.
3. Document structured-output limitations clearly: many HuggingFace-hosted models do not support the same structured/JSON-mode guarantees as OpenAI, and this must not be silently assumed to work.
4. Document tool-calling limitations similarly — only enable tool-calling paths for models/endpoints confirmed to support them.
5. Add authentication handling for the HuggingFace API token via the existing secret-management approach.

6. Add clear, specific error handling for rate limits and model-loading delays, which are common with hosted inference endpoints.

7. Expected Code Changes

- HuggingFaceLLMProvider implementation satisfying the existing provider interface.

8. Database Changes

- None.

9. API Changes

- None.

10. Worker / Background-Job Changes

- None directly.

11. AI / LLM Changes

- Response quality and structured-output reliability will vary significantly by model and must be evaluated per model before any production use.

12. Security Considerations

- Store the HuggingFace API token using the existing secret-management mechanism; never hardcode it.
- If using a self-hosted endpoint, ensure it is not publicly exposed without access control.

13. Error-Handling Requirements

- Rate-limit responses → backoff and retry with a bounded number of attempts.
- Model-loading/cold-start delays (common on hosted inference) → surface a clear, distinct error rather than a generic timeout.
- Unsupported structured output or tool calling for the configured model → fail clearly rather than returning malformed data.

14. Testing Strategy

- Unit test: HuggingFaceLLMProvider conforms to the shared provider interface.
- Integration test (requires credentials/endpoint): a basic chat completion succeeds.
- Test documenting behavior when structured output is requested from a model that does not support it.

15. Manual Verification Steps

1. Configure valid HuggingFace credentials/endpoint and switch the active provider.
2. Send a simple test conversation and confirm a coherent response.
3. Attempt a structured-output request and confirm the documented behavior (success or clear failure) matches what was implemented.

16. Definition of Done

- HuggingFaceLLMProvider implements the shared provider interface.
- Structured-output and tool-calling capabilities/limitations are documented per tested model/endpoint.
- Core platform tests still pass with OpenAI as the default provider.

17. Common Mistakes

- Assuming structured output/tool calling works uniformly across all HuggingFace-hosted models.
- Not handling model cold-start latency, leading to confusing timeout errors.
- Hardcoding a single model name instead of making it configurable.

18. Rollback / Recovery Approach

As with Ollama, provider selection is configuration-driven; reverting to OpenAI requires only a configuration change.

19. What Should NOT Be Changed

- Do not modify the shared provider interface to work around HuggingFace-specific limitations — handle them inside this provider's implementation.

20. What The Next Phase Depends On

PHASE 11 (production hardening) applies across all providers and all phases completed so far, and should only begin once the team is confident the core (Phases 0–8) is stable.

PHASE 11

Testing, Security, Observability, and Production Hardening

1. Objective

Systematically harden the platform across security, reliability, observability, and testing before it is trusted with real production traffic and customer data.

2. Current Problem

Individual phases above each include their own security/testing requirements, but no phase so far addresses cross-cutting, platform-wide concerns such as rate limiting, SSRF protection, dead-letter handling, or end-to-end observability. Without this phase, the platform may work functionally but remain fragile or unsafe under real-world conditions. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.** for the current state of each item below, since some may already be partially implemented.

3. Why This Phase Is Needed

This phase is deliberately last: hardening work is most effective once the real shape of the system (agents, tools, workers, APIs) is stable, so security and reliability controls are applied to final, not moving, targets.

4. Dependencies

- All prior phases the team intends to ship should be functionally complete and manually verified before broad hardening work begins, though individual hardening items (e.g., rate limiting on an existing endpoint) can be pulled forward if a real risk is identified earlier.

5. Files / Modules Likely Involved

- Spans most of the codebase: API routers, worker tasks, agent/tool layer, configuration, and deployment (Docker) files. **VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.**

6. Step-by-Step Implementation Tasks

1. Security pass: confirm authentication and workspace-authorization are enforced on every endpoint, not just the ones built in earlier phases; confirm webhook signature verification is applied to the WhatsApp webhook; confirm secrets are never logged or hardcoded.
2. Add or verify rate limiting on public-facing endpoints (especially the WhatsApp webhook and any knowledge-upload endpoint) to prevent abuse.
3. Add SSRF protection for any feature that fetches a user-supplied URL (e.g., URL-based knowledge ingestion) — block requests to internal/private IP ranges and metadata endpoints.
4. Review prompt-injection exposure across all places where external content (documents, tool results, web content) enters the LLM context, and confirm such content is never treated as an instruction.
5. Reliability pass: confirm sensible timeouts and bounded retries exist for external calls (LLM provider, CRM, MCP tools, WhatsApp API); confirm ARQ job failures are handled (dead-letter or clear failure state) rather than silently dropped.

6. Confirm database operations that must be atomic (e.g., lead creation + CRM sync trigger) use appropriate transaction boundaries.

7. Observability pass: confirm structured logs (structlog) include correlation IDs, conversation IDs, and agent run IDs so a single request can be traced end-to-end; confirm OpenTelemetry spans cover the webhook → worker → agent → tool → response path; add Prometheus metrics for latency, error rate, and token usage if not already present.

8. Testing pass: fill gaps across unit, integration, API, worker, agent, RAG, security, multimodal, and end-to-end test suites, prioritizing the phases most recently implemented.

7. Expected Code Changes

- Rate-limiting middleware/dependency applied to relevant endpoints.
- SSRF-safe URL fetching utility used wherever user-supplied URLs are processed.
- Consistent timeout/retry wrappers around external calls.
- Correlation ID propagation through logs and traces.

8. Database Changes

- Possible addition of an audit_logs table if not already introduced in Phase 8; otherwise none expected.

VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.

9. API Changes

- No new business endpoints; changes are cross-cutting (rate limiting, validation) applied to existing endpoints.

10. Worker / Background-Job Changes

- Dead-letter/error-handling behavior added or confirmed for all ARQ tasks introduced in earlier phases.

11. AI / LLM Changes

- Systematic review of every point where external or user-supplied content reaches the LLM, to confirm prompt-injection safeguards are applied consistently.

12. Security Considerations

- Authentication and workspace isolation enforced platform-wide.
- Secret management reviewed for every credential (OpenAI, HubSpot, WhatsApp, Ollama/HuggingFace tokens).
- Webhook signature verification confirmed on the WhatsApp webhook.
- Rate limiting on public endpoints.
- Input and file validation confirmed on every upload path.
- SSRF protection on any user-supplied URL fetch.
- Prompt-injection review across all agent/tool content boundaries.
- Tool security (permissions, credential isolation) confirmed per Phase 8.

13. Error-Handling Requirements

- Every external call (LLM, CRM, MCP, WhatsApp API) has a timeout and a bounded retry policy.
- Every ARQ task has a defined failure/dead-letter behavior instead of silent job loss.
- Database operations that must be atomic use transactions with proper rollback on failure.

14. Testing Strategy

- Unit tests across all modules touched in Phases 2–10.
- Integration tests for each worker task's end-to-end behavior.
- API tests covering authentication, authorization, and validation for every endpoint.
- Agent/RAG tests covering retrieval quality and multi-turn conversation handling.
- Security tests: attempted cross-workspace access, prompt-injection payloads, oversized/invalid uploads, SSRF attempts against URL ingestion.
- Multimodal tests: WhatsApp document, image, and audio handling paths.
- End-to-end tests simulating a full customer conversation from webhook to response.

15. Manual Verification Steps

1. Run a full end-to-end scenario (text conversation, document upload, lead capture, CRM sync) against a staging environment and confirm traces/logs allow the whole path to be followed by one correlation ID.
2. Attempt at least one deliberate misuse test per security item above (e.g., call an endpoint without auth, submit an oversized file, submit a private-network URL) and confirm each is correctly blocked.
3. Review Prometheus/OpenTelemetry dashboards (if available) for latency and error-rate visibility across the full pipeline.

16. Definition of Done

- Every item in the security, reliability, observability, and testing checklists above has been reviewed and is either confirmed already in place or implemented.
- Deliberate misuse tests are blocked as expected.
- A real request can be traced end-to-end through logs/traces using a single correlation ID.
- Test suites across all categories listed above are passing in CI.

17. Common Mistakes

- Treating hardening as a single checklist to rush through at the end rather than verifying each item concretely against the real system.
- Adding rate limiting or timeouts with arbitrary values never validated against real usage patterns.
- Assuming observability instrumentation exists because the libraries (structlog, OpenTelemetry) are installed, without confirming it is actually wired into the request path.
- Skipping negative/misuse testing and only testing the happy path.

18. Rollback / Recovery Approach

Hardening changes should be applied incrementally and independently (e.g., rate limiting on one endpoint at a time) so that any single change causing a regression can be reverted without affecting unrelated hardening work.

19. What Should NOT Be Changed

- Do not use this phase as justification for a broad rewrite of earlier phases — hardening should tighten and verify existing behavior, not redesign it.

20. What The Next Phase Depends On

This is the final planned phase; its completion is the basis for considering the platform production-ready for the scope defined in this roadmap.

Master Checklist

This checklist lists every task in dependency order. Each task follows the same four-step confirmation pattern before it is considered complete:

☐ Task
☐ Verify
☐ Test
☐ Mark complete

PHASE 0 — Verify repository against audit claims

Dependency: None | **Test required:** N/A (verification)

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 1 — Verify PostgreSQL, Redis, Alembic, FastAPI, ARQ, env vars, OpenAI

Dependency: PHASE 0 | **Test required:** Manual checks

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 2 — Load real conversation history into AgentState

Dependency: PHASE 1 | **Test required:** Unit + integration

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 3 — Implement ingest_knowledge_source worker task

Dependency: PHASE 1 | **Test required:** Unit + integration

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 4 — Build Knowledge Source Upload API

Dependency: PHASE 3 | **Test required:** API tests

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 5 — Implement `process_document_message` (WhatsApp docs)

Dependency: PHASE 3 | **Test required:** Unit + integration

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 6 — Build Workspace/Admin API

Dependency: PHASE 4 | **Test required:** API tests

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 7 — Implement `sync_lead_to_crm` (Mock + HubSpot)

Dependency: PHASE 1 | **Test required:** Unit + integration

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 8 — Establish MCP + governance/tool-security layer

Dependency: PHASE 7 | **Test required:** Unit + integration + security

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 9 — Implement `OllamaLLMProvider` (optional)

Dependency: Core phases stable | **Test required:** Unit + integration

- ☐ Task implemented
- ☐ Verified against real repository behavior

- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 10 — Implement HuggingFaceLLMProvider (optional)

Dependency: PHASE 9 pattern | **Test required:** Unit + integration

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

PHASE 11 — Security, reliability, observability, testing hardening

Dependency: All prior phases | **Test required:** Full test suite + security tests

- ☐ Task implemented
- ☐ Verified against real repository behavior
- ☐ Automated/manual tests pass
- ☐ Marked complete and documented

Priority & Status Table

Use this table to track progress. Update the Status column as work proceeds using: NOT STARTED, IN PROGRESS, BLOCKED, TESTING, COMPLETED.

Pri.	Phase	Task	Dependency	Main Files	Test Req.	Status
1	P0	Verify audit claims against real repository	None	Whole repo (read-only)	No (verification)	NOT STARTED
2	P1	Verify infrastructure (DB, Redis, Alembic, FastAPI, ARQ, env, OpenAI)	P0	Config/env, DB, worker startup	Manual checks	NOT STARTED
3	P2	Load real conversation history into AgentState	P1	Agent entry point, message/conversation models (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes	NOT STARTED
4	P3	Implement ingest_knowledge_source	P1	Worker task, KnowledgeSource/DocumentChunk models (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes	NOT STARTED
5	P4	Knowledge Source Upload API	P3	FastAPI router, schemas, auth deps (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes	NOT STARTED
6	P5	process_document_message (WhatsApp docs)	P3	Worker task, webhook handler, OCR (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes	NOT STARTED
7	P6	Workspace/Admin API	P4	FastAPI router, BusinessProfile model (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes	NOT STARTED
8	P7	sync_lead_to_crm (Mock + HubSpot)	P1	Worker task, Lead model, CRM provider interface (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes	NOT STARTED
9	P8	MCP + governance/tool-security layer	P7	Tool registry, governance layer, MCP server dir (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes	NOT STARTED
10	P9	OllamaLLMProvider (optional)	Core stable	Provider factory/interface (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes	NOT STARTED
11	P10	HuggingFaceLLMProvider (optional)	P9 pattern	Provider factory/interface (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes	NOT STARTED

Pri.	Phase	Task	Depend ency	Main Files	Test Req.	Status
12	P11	Security / reliability / observability / testing hardening	All prior	Cross-cutting (VERIFY IN REPOSITORY BEFORE IMPLEMENTATION.)	Yes (full suite)	NOT STARTED

START HERE

If nothing else in this document is read first, read this section. These are the only five actions the developer should take before doing anything else. The first action is verification, not implementation.

1. Verify the repository against this document's assumptions

Open the actual codebase and check whether each reported item (`process_document_message`, `ingest_knowledge_source`, `sync_lead_to_crm`, `OllamaLLMProvider`, `HuggingFaceLLMProvider`, stale workflow comments, empty MCP directory, conversation history, knowledge source upload API, workspace/admin API) matches its audit classification. Record real findings using PHASE 0's checklist and status labels. Do not write any feature code yet.

2. Verify infrastructure is running

Confirm PostgreSQL and Redis are reachable, Alembic migrations are up to date, and FastAPI/ARQ start cleanly using the real environment configuration, following PHASE 1's exact checks.

3. Confirm which mandatory external credentials are configured

Check whether an OpenAI API key is set (or an alternative provider is fully configured) and whether Meta WhatsApp credentials are available for real-channel testing. Note what is missing rather than assuming it will appear later.

4. Read PHASE 2 in full before writing any code

Conversation history is the first implementation task in this roadmap because nearly everything else depends on the agent having real context. Understand the current `run_agent` / `AgentState` wiring before changing it.

5. Apply the UNDERSTAND → INSPECT → DESIGN → IMPLEMENT → TEST → VERIFY → FIX → DOCUMENT cycle to the very first task

Do not skip INSPECT EXISTING CODE or MANUAL VERIFY, even for a task that looks simple. This sets the working pattern for every phase that follows.

Once these five actions are complete, proceed to PHASE 2 in full, then PHASE 3, following the phase order defined in this document. Do not skip ahead to later phases (e.g., MCP or optional providers) before the core phases are stable.